

SIMATS

ENGINEERING Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

1.8 bit addition and subtraction

- Number 1 = -75
- Number 2 = $+38$
- Representation = 8-bit 2's Complement

Step 1: Convert the Numbers to Binary

(a) Represent $+75$

$75 = 01001011$

(b) Represent -75

Take 2's complement of 01001011:

- 1's complement = 10110100
- Add 1 = 10110101

$-75 = 10110101$

(c) Represent $+38$

$38 = 00100110$

1. Addition ($-75 + 38$)

```
10110101
+ 00100110
-----
11011011
```

Result in Binary: 11011011

Since MSB = 1, the result is negative.

Find decimal value:

- 1's complement = 00100100
- Add 1 = 00100101 = 37

Therefore,

Result = -37

Overflow Check

- Adding one negative and one positive number cannot cause overflow.

Overflow = No

Final Answer

- Binary = 11011011
- Decimal = -37

2. Subtraction (-75 - 38)

Subtraction is performed as:

$$-75 + (-38)$$

Find 2's complement of +38

$$+38 = 00100110$$

- 1's complement = 11011001
- Add 1 = 11011010

So,

$$-38 = 11011010$$

Now add:

```
  10110101
+ 11011010
-----
```

1 10001111

Result = 10001111

Interpret the Result

Since MSB = 1, it is negative.

Take 2's complement:

- 1's complement = 01110000
- Add 1 = 01110001 = 113

Therefore,

Result = -113

Overflow Check

Adding two negative numbers should give a negative result.

The mathematical answer (-113) lies within the 8-bit 2's complement range (-128 to +127), so:

Overflow = No

Final Results

Operation	Binary Result	Decimal Result	Overflow
-----------	---------------	----------------	----------

-75 + 38	11011011	-37	No
----------	----------	-----	----

-75 - 38	10001111	-113	No
----------	----------	------	----

2. Carry Look-Ahead Adder (CLA) – 8-bit

Given Operand:

- **A = 10110110**
- **B = 01101101**

For each bit:

- **Generate (Gi) = $A_i \times B_i$** (AND operation)
- **Propagate (Pi) = $A_i \oplus B_i$** (XOR operation)

Bit	A	B	G = A·B	P = A $\oplus$ B
0	0	1	0	1
1	1	0	0	1
2	1	1	1	0
3	0	1	0	1
4	1	0	0	1
5	1	1	1	0
6	0	1	0	1
7	1	0	0	1

Assume initial carry **C₀ = 0**

The carry equations are:

- $C_1 = G_0 + P_0C_0$
- $C_2 = G_1 + P_1G_0 + P_1P_0C_0$
- $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
- ...
- C_8 is calculated similarly.

Unlike a Ripple Carry Adder, all carry bits are computed simultaneously, reducing delay.

The sum bits are calculated using:

$$S_i = P_i \oplus C_i$$

Thus,

- $S_0 = P_0 \oplus C_0$
- $S_1 = P_1 \oplus C_1$
- ...
- $S_7 = P_7 \oplus C_7$

The final result is obtained much faster because carries are already available.

Comparison: Ripple Carry Adder (RCA) vs Carry Look-Ahead Adder (CLA)

Feature	Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry Generation	Sequential	Simultaneous
Speed	Slower	Very Fast
Delay	Increases with number of bits	Nearly Constant
Hardware	Simple	More Complex
Cost	Low	Higher
Performance	Moderate	High
Used In	Small circuits	Modern processors and CPUs

Execution Time Comparison

Ripple Carry Adder

- Carry travels from one stage to the next.
- Total delay ≈ 8 full-adder delays for an 8-bit adder.

Carry Look-Ahead Adder

- Carries are generated in parallel using logic gates.
- Delay is much lower CLA is significantly faster than RCA

Why CLA is Widely Used in Modern Processors

1. High-speed arithmetic operations.
2. Carry bits are generated simultaneously, reducing propagation delay.
3. Improves CPU performance in ALUs.
4. Suitable for high-frequency processors.
5. Reduces execution time for arithmetic instructions.

The Carry Look-Ahead Adder (CLA) improves addition speed by computing generate (G) and propagate (P) signals and calculating all carry bits simultaneously. Although it requires more hardware and complex logic than the Ripple Carry Adder, its high speed and low delay make it the preferred choice in modern processors and high-performance digital systems.

3. Booth's Algorithm (Signed Multiplication)

Given

- Multiplicand (M) = +13
- Multiplier (Q) = -6

Step 1: Binary Representation

- +13 = 01101
- -13 = 10011 (2's complement of 01101)
- -6 = 11010

Initial values:

- A (Accumulator) = 00000
- Q (Multiplier) = 11010
- Q_{-1} (Booth bit) = 0
- M = 01101
- -M = 10011

Booth's Rules

- $Q_0Q_{-1} = 00 \rightarrow$ No operation

- $Q_0Q_{-1} = 01 \rightarrow A = A + M$
- $Q_0Q_{-1} = 10 \rightarrow A = A - M$
- $Q_0Q_{-1} = 11 \rightarrow \text{No operation}$

After each operation, perform Arithmetic Right Shift (ARS).

Iteration Table

Iteration	Q_0Q_{-1}	Operation	A	Q	Q_{-1}
Initial	–	–	00000	11010	0
1	00	No operation $\rightarrow$ ARS	00000	01101	0
2	10	$A = A - M$ ARS	10011 11001	01101 10110	0 1
3	01	$A = A + M$ ARS	00110 00011	10110 01011	1 0
4	10	$A = A - M$ ARS	10110 11011	01011 00101	0 1
5	11	No operation $\rightarrow$ ARS	11101	10010	1

Final Product

Combine A and Q:

1110110010

This is a 10-bit 2's complement number.

Convert to decimal:

- Decimal value = -78
 $13 \times (-6) = -78$

The Arithmetic Right Shift:

- Shifts all bits one position to the right.
- Preserves the sign bit (MSB).
- Maintains the correct value for signed numbers in 2's complement.
- Combines the effects of multiplication and division by 2 during Booth's algorithm.

How Booth's Algorithm Reduces Addition/Subtraction

Booth's algorithm examines two bits (Q_0 and Q_{-1}) instead of every bit individually.

- Consecutive 1's in the multiplier are treated as a single operation.
- It performs one addition and one subtraction for a block of 1's rather than multiple additions.
- This significantly reduces the number of arithmetic operations, especially when the multiplier contains long sequences of 1's.

Example:

- Multiplier 11110
 - Normal multiplication: several additions.
 - Booth's algorithm: only one subtraction and one addition.

Advantages of Booth's Algorithm

- Efficient for signed binary multiplication.
- Reduces the number of addition and subtraction operations.
- Handles positive and negative numbers directly.
- Faster than ordinary shift-and-add multiplication when multipliers have consecutive 1's.
- Widely used in processors, ALUs, and digital signal processors (DSPs).

Final Answer

- Multiplicand (M) = +13 = 01101 and Multiplier (Q) = -6 = 11010
- Final Product (Binary) = 1110110010
- Final Product (Decimal) = -78

4. Division Using Restoring and Non-Restoring Algorithms

Given

- Dividend = 27
- Divisor = 5

Binary representation (5 bits):

- Dividend (Q) = 11011_2 (27)
- Divisor (M) = 00101_2 (5)

1. Restoring Division Algorithm

Steps

1. Left shift (A,Q).
2. $A = A - M$.
3. If $A < 0$:
 - Restore $A = A + M$.
 - Quotient bit = 0.
4. Else:
 - Quotient bit = 1.

Iteration Table

Iteration A after Subtraction Action Quotient (Q)

Initial	00000	–	11011
1	Negative	Restore	10110
2	Positive	$Q_0 = 1$	01101
3	Negative	Restore	11010
4	Positive	$Q_0 = 1$	10101
5	Positive	$Q_0 = 1$	00101

Final Answer:

- Quotient = $00101_2 = 5$
- Remainder = $00010_2 = 2$

2. Non-Restoring Division Algorithm

Steps

1. Left shift (A,Q).
2. If previous remainder is positive:
 - $A = A - M$
3. If previous remainder is negative:
 - $A = A + M$
4. Set quotient bit according to sign of A.
5. Restore only once at the end if required.

Iteration Table

Iteration Operation Remainder (A) Quotient (Q)

Initial	–	00000	11011
1	$A - M$	Negative	10110
2	$A + M$	Positive	01101
3	$A - M$	Negative	11010
4	$A + M$	Positive	10101
5	$A - M$	Positive	00101

Final Result

- Quotient = $00101_2 = 5$
- Remainder = $00010_2 = 2$

Comparison of Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration	Required after every negative remainder	Only one final correction if needed
Number of Operations	More	Fewer
Execution Speed	Slower	Faster
Hardware Complexity	Higher	Lower
Efficiency	Moderate	High
Processor Usage	Older processors	Modern processors

Computational Complexity

Restoring Division

- Performs subtraction and sometimes an additional restoration.
- Requires more arithmetic operations.
- Longer execution time.

Non-Restoring Division

- Eliminates repeated restoration.
- Performs either addition or subtraction in each iteration.
- Reduces total operations.

Hardware Efficiency

Restoring Division

- Needs extra circuitry for restoration.
- More control logic.
- Higher hardware cost.

Which Algorithm is Better for High-Speed Processors?

Non-Restoring Division is more suitable because:

- It avoids repeated restoration steps.
- Executes fewer arithmetic operations.
- Provides faster execution.
- Requires simpler hardware.
- Improves processor performance and throughput.

Final Answer

Restoring Division

- Quotient = 00101_2 (5)
- Remainder = 00010_2 (2)

Non-Restoring Division

- Quotient = 00101_2 (5)
- Remainder = 00010_2 (2)

5. IEEE 754 Floating-Point Representation

Given Number

Decimal Number = -42.375

Step 1: Convert Decimal to Binary

Integer Part

$$42_{10} = 101010_2$$

Fractional Part

$$0.375 \times 2 = 0.75 \rightarrow 0$$

$$0.75 \times 2 = 1.5 \rightarrow 1$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

Therefore,

$$0.375_{10} = 0.011_2$$

$$42.375_{10} = 101010.011_2$$

Normalize:

$$101010.011_2 = 1.01010011_2 \times 2^5$$

1. IEEE 754 Single Precision (32-bit)

Sign Bit

Negative number

$$S = 1$$

Exponent

$$\text{Bias} = 127$$

$$\text{Actual exponent} = 5$$

$$\text{Stored exponent} = 5 + 127 = 132$$

$$132_{10} = 10000100_2$$

Remove the leading 1:

010100110000000000000000

Final 32-bit Representation

Sign Exponent Mantissa

1 10000100 010100110000000000000000

Binary:

1 10000100 010100110000000000000000

2. IEEE 754 Double Precision (64-bit)

$$S = 1$$

$$\text{Bias} = 1023$$

$$= 1023 + 5$$

$$= 1028$$

$$1028_{10} = 10000000100_2$$

01010011000

Final 64-bit Representation

Sign	Exponent	Mantissa
------	----------	----------

```
1    10000000100 01010011000000000000000000000000000000000000000000
```

Floating-Point Addition

Add

$$-42.375 + 15.125$$

Step 1: Convert Both Numbers

-42.375

Binary

$$1.01010011 \times 2^5$$

+15.125

$$15 = 1111_2$$

$$0.125 = .001_2$$

$$15.125 =$$

$$1111.001_2$$

Normalize

$$1.111001 \times 2^3$$

Step 2: Exponent Alignment

Largest exponent = 5

Shift the smaller mantissa right by 2 bits

$$1.111001 \times 2^3$$

↓

$$0.01111001 \times 2^5$$

Now both exponents are 5.

Step 3: Mantissa Operation

Since the signs are different, subtract the mantissas.

1.01010011

-0.01111001

0.11011010

Step 4: Normalization

Normalized result:

1.1011010×2^4

Exponent decreases from 5 to 4 after shifting the mantissa left by one position.

Step 5: Rounding

The mantissa is rounded according to the IEEE 754 default mode (Round to Nearest, Even).

In this example, no significant rounding is needed because the value is exactly representable.

Step 6: Final Decimal Result

$-42.375 + 15.125$

$= -27.25$

Binary:

11011.01_2

Normalized:

1.101101×2^4

Impact of Precision

Single Precision (32-bit)

- 23-bit mantissa
- About 7 decimal digits of precision
- Faster execution
- Small rounding errors may occur

Double Precision (64-bit)

- 52-bit mantissa

- About 15–16 decimal digits of precision
- Much higher accuracy
- Lower rounding error
- Preferred for scientific and engineering calculations

Comparison

Feature	Single Precision	Double Precision
Total Bits	32	64
Sign Bit	1	1
Exponent Bits	8	11
Mantissa Bits	23	52
Bias	127	1023
Precision	~7 decimal digits	~15–16 decimal digits
Memory Usage	Less	More
Accuracy	Moderate	High

Final Answer

IEEE 754 (32-bit)

- Sign: 1
- Exponent: 10000100
- Mantissa: 010100110000000000000000

IEEE 754 (64-bit)

- Sign: 1
- Exponent: 10000000100
- Mantissa: 01010011000

Floating-Point Addition

- Operation: $-42.375 + 15.125$
- Exponent Alignment: Smaller exponent shifted right by 2 bits.
- Mantissa Operation: Subtraction (different signs).
- Normalized Result: 1.101101×2^4
- Final Decimal Result: -27.25

IEEE 754 stores floating-point numbers using sign, exponent, and mantissa. During addition, the exponents are first aligned, the mantissas are added or subtracted based on the signs, the result is normalized and rounded, and the final value is stored. Double precision provides significantly greater accuracy than single precision due to its larger exponent and mantissa fields.